Due Date 4/30/26

Group Members: Ziyad, Gabe, Mahamudurdaiyan, Jake

Theory of Automata Final Project Report

Section 1. NFA/DFA Implementation:

Our group decided to implement the NFA.

Our NFA simulator is designed to handle non-deterministic transitions by exploring all possible computation paths. Through this project we learned how computers are able to recognize and handle decision making based on human language. We came to appreciate the beauty of Automata and Turing Machines. Using a stack-based backtracking implementation to our code, the program thoroughly checks every valid branch for a given input string.

This ensures that if at least one path leads to an accept state, the machine correctly identifies the string as "Accepted." If at least one path leads to a reject state, the machine identifies the string as “rejected”. If the user enters nothing, the machine identifies as an empty set and terminates.

Section 3. Data Structures Implemented:

Tuples for individual configs and transitions, stacks using lists to explore NFA branches, dictionaries to store full NFA , lists for test strings, tokens, list for results, sets for alphabet, states, accept states, one item list pos = [0], tracks parser position.

A. The NFA Machine (Dictionary) • Alphabet (sigma): A Set of symbols. This defines the valid characters the machine can process.

• States (states): A Set of unique string identifiers representing every possible state in the machine.

• Start State (start): A single String variable pointing to the initial state.

• Accept States (accept): A Set of strings. Using a set allows for average time complexity when checking if a final state is reached.

• Transitions (transitions): A List of Tuples. Each tuple (from, sym, to) represents a move from one state to another on a specific symbol. This format directly supports non-determinism by allowing multiple "to" states for the same "from/sym" pair. B. Simulation & Parsing Structures

• **Backtracking Stack:** A LIFO (Last-In, First-Out) list. Each element is a configuration tuple (state, remaining_string). This tracks the different paths the NFA can take simultaneously.
• **Mutable Parser Pointer:** A List containing a single integer [pos]. Because integers are immutable in Python, wrapping the index in a list allows the nested parsing functions to update the global reading position in the file.
• **Input Strings: Immutables Strings;** The input is processed using slicing (remaining[1:]), which creates a new string for each step of the simulation, ensuring the original input remains unchanged for other branches.

Section 4. Algorithm Pseudocode:

Main(no args)

asks user input for file name
try -catch to open the file
if file “tests” lists has items go through the the items in test and puts them as args in simulate and if simulate returns true, “accepted” is appended to list results if not “rejected” is appended.
print results
if “tests” has no items
ask user to input single string to check against the NFA

Simulate(nfa,input string)

pushes start state and full input
while the stack isn't empty
pop a configuration(made op of state and remaining)
if at final state and no remaining digits return true as an accepted string
else continue through the nodes of the NFA
Read the next character in the string
for (from state, symbol, to state) in NFA[transitions] //go through //computations
if frm = state and sym = symbol //from state is equal to state in the //stack and the symbol got from the stack is equal to the symbol in the //computation
add the next possible state(next state)and the remaining digits in the string to the stack
If no solution is found return false

Section 5. Test Cases:

We have 2 modes for both input files provided:
Batch Mode Testing Machine File: proj-1-machine.txt Description: NFA representing strings that contain the substring 01. Test Case 1: 1101 Result: Accepted Technical Analysis: The stack correctly explores the path where the machine stays in q0 for the first two 1s, moves to q1 on 0, and reaches q2 on the final 1. Test Case 2: 0001 Result: Accepted
Technical Analysis: Proves the NFA can handle repeated symbols in the start state before transitioning to the accept state. Test Case 3: 1110 Result: Rejected Technical Analysis: The stack exhausts all paths and finds no sequence that ends in the accept state q2. Program Console Output: (accepted, accepted, rejected) Interactive Mode
Testing Machine File: interactive_test.txt (NFA with empty test tuple "()") User Prompt: Please input a string: User Input: 1101 Program Output: Accepted. User Prompt: Please input another string: User Input: 000 Program Output: Rejected.
User Prompt: Please input another string: User Input: (Press Enter/Blank) Program Output: Bye bye. (Program terminates correctly)
Example output test case from terminal:

```
Please input the file name:  project-1-machine-1.txt
Please input a string:  100001

Accepted.

Please input another string:  0001

Accepted.

Please input another string:  010101010011

Rejected.

Please input another string:  011

Rejected.

Please input another string:  01

Accepted.

Please input another string:  10

Rejected.

Please input another string:  1000000000000

Rejected.

Please input another string:  011

Rejected.

Please input another string:  01

Accepted.

Please input another string:  1

Rejected.

Please input another string:  10
```

```
Rejected.

Please input another string:  001

Accepted.

Please input another string:  00000011

Rejected.

Please input another string:  101

Accepted.

Please input another string:  0000

Rejected.

Please input another string:  100000

Rejected.

Please input another string:  101

Accepted.

Please input another string:

Bye bye.
```

Section 6. Individual Contributions:

We all collaborated on the main code and structure,

Jake- Pseudocode implementation, data structure implementation in code,

Ziyad- Report and Readme, test cases part 1,

Moe- code implementation and correctness,

Gabe- Report and Readme.txt, test cases part 2,